

Phụ lục D. Đồ thị TensorFlow

Trong phụ lục này, chúng ta sẽ khám phá các đồ thị được tạo bởi các hàm TF (xem Chương 12).

Hàm TF và Hàm Cụ thể (Concrete Functions)

Các hàm TF là đa hình (polymorphic), nghĩa là chúng hỗ trợ đầu vào thuộc các kiểu (và hình dạng) khác nhau. Ví dụ, hãy xem xét hàm `tf_cube()` sau:

```
import tensorflow as tf

@tf.function
def tf_cube(x):
    return x**3
```

Mỗi khi bạn gọi một hàm TF với một sự kết hợp mới của các kiểu hoặc hình dạng đầu vào, nó sẽ tạo ra một **hàm cụ thể (concrete function)** mới, với đồ thị riêng của nó được chuyên biệt hóa cho sự kết hợp cụ thể này. Sự kết hợp các kiểu và hình dạng đối số như vậy được gọi là một **chữ ký đầu vào (input signature)**. Nếu bạn gọi hàm TF với một chữ ký đầu vào mà nó đã thấy trước đó, nó sẽ sử dụng lại hàm cụ thể mà nó đã tạo trước đó. Ví dụ, nếu bạn gọi `tf_cube(tf.constant(3.0))`, hàm TF sẽ sử dụng lại cùng hàm cụ thể mà nó đã sử dụng cho `tf_cube(tf.constant(2.0))` (đối với các tensor vô hướng float32). Nhưng nó sẽ tạo ra một hàm cụ thể mới nếu bạn gọi `tf_cube(tf.constant([2.0]))` hoặc `tf_cube(tf.constant([3.0]))` (đối với các tensor float32 có hình dạng [1]), và một hàm khác nữa cho `tf_cube(tf.constant([[1.0, 2.0], [3.0, 4.0]]))` (đối với các tensor float32 có hình dạng [2, 2]). Bạn có thể lấy hàm cụ thể cho một sự kết hợp đầu vào cụ thể bằng cách gọi phương thức `get_concrete_function()` của hàm TF. Sau đó, nó có thể được gọi như một hàm thông thường, nhưng nó sẽ chỉ hỗ trợ một chữ ký đầu vào (trong ví dụ này, các tensor vô hướng float32):

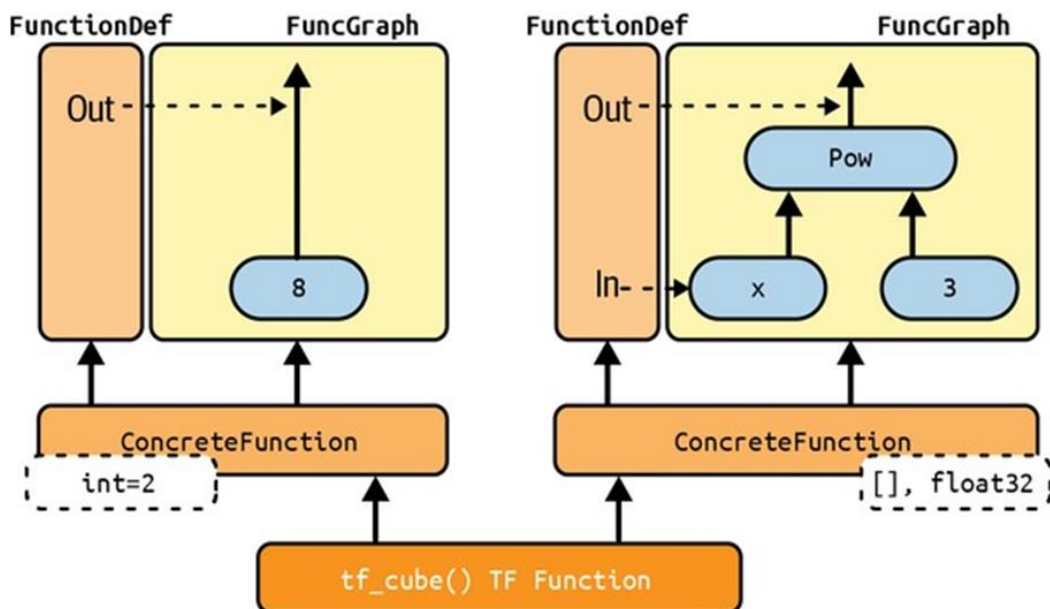
```
>>> concrete_function = tf_cube.get_concrete_function(tf.constant(2.0))

>>> concrete_function
<ConcreteFunction tf_cube(x) at 0x...> # Địa chỉ có thể khác

>>> concrete_function(tf.constant(2.0))
<tf.Tensor: shape=(), dtype=tf.float32, numpy=8.0>
```

Hình D-1 cho thấy hàm TF `tf_cube()`, sau khi chúng ta gọi `tf_cube(2)` và `tf_cube(tf.constant(2.0))`: hai hàm cụ thể đã được tạo, một cho mỗi chữ ký, mỗi hàm có đồ thị hàm tối ưu hóa riêng (FuncGraph) và định nghĩa hàm riêng (FunctionDef). Một định nghĩa hàm trỏ đến các phần của đồ thị tương ứng với đầu vào và đầu ra của hàm. Trong mỗi FuncGraph, các nút (hình bầu dục) biểu thị các phép toán (ví dụ: lũy thừa, hằng số, hoặc các placeholder cho các đối số như x), trong khi các cạnh (các mũi tên liền nét giữa các phép toán) biểu thị các tensor sẽ chảy qua đồ thị. Hàm cụ thể bên trái được chuyên biệt hóa cho $x = 2$, vì vậy TensorFlow đã quản lý để đơn giản hóa nó chỉ để xuất ra 8 mọi lúc (lưu ý rằng

định nghĩa hàm thậm chí không có đầu vào). Hàm cụ thể bên phải được chuyên biệt hóa cho các tensor vô hướng float32, và nó không thể được đơn giản hóa. Nếu chúng ta gọi `tf_cube(tf.constant(5.0))`, hàm cụ thể thứ hai sẽ được gọi, phép toán placeholder cho x sẽ xuất ra 5.0, sau đó phép toán lũy thừa sẽ tính 5.0^3 , vì vậy đầu ra sẽ là 125.0.



Hình D-1. Hàm TF `tf_cube()`, với các Hàm Cụ thể và FuncGraph của chúng.

Các tensor trong các đồ thị này là các **tensor ký hiệu (symbolic tensors)**, nghĩa là chúng không có giá trị thực tế, chỉ có kiểu dữ liệu, hình dạng và tên. Chúng biểu thị các tensor trong tương lai sẽ chảy qua đồ thị một khi một giá trị thực tế được đưa vào placeholder x và đồ thị được thực thi. Các tensor ký hiệu giúp có thể chỉ định trước cách kết nối các phép toán, và chúng cũng cho phép TensorFlow tự động suy luận kiểu dữ liệu và hình dạng của tất cả các tensor một cách đệ quy, dựa trên kiểu dữ liệu và hình dạng của đầu vào của chúng.

Bây giờ hãy tiếp tục tìm hiểu sâu hơn, và xem cách truy cập định nghĩa hàm và đồ thị hàm và cách khám phá các phép toán và tensor của đồ thị.

Khám phá Định nghĩa Hàm và Đồ thị

Bạn có thể truy cập đồ thị tính toán của một hàm cụ thể bằng thuộc tính `graph`, và lấy danh sách các phép toán của nó bằng cách gọi phương thức `get_operations()` của đồ thị:

```
>>> concrete_function.graph
<tensorflow.python.framework.func_graph.FuncGraph object at 0x...> # Địa chỉ có thể khác

>>> ops = concrete_function.graph.get_operations()

>>> ops
[<tf.Operation 'x' type=Placeholder>,
 <tf.Operation 'pow/y' type=Const>]
```

```
<tf.Operation 'pow' type=Pow>,
<tf.Operation 'Identity' type=Identity>]
```

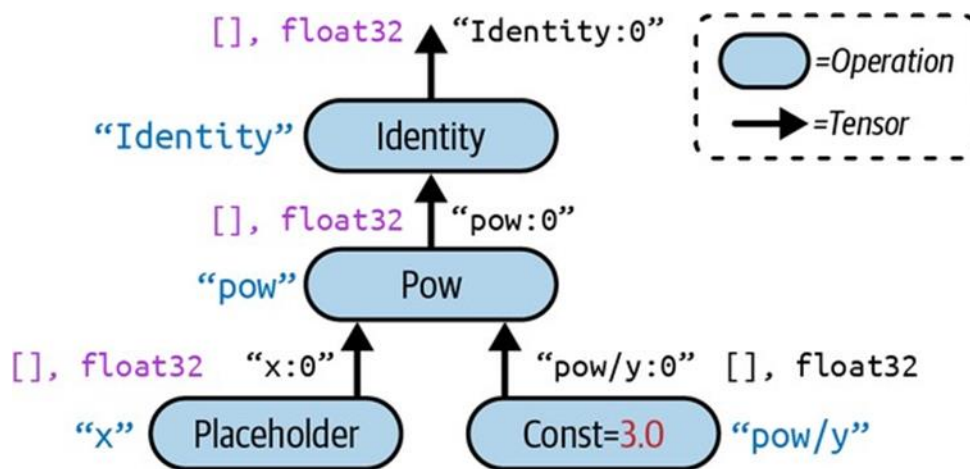
Trong ví dụ này, phép toán đầu tiên biểu thị đối số đầu vào x (nó được gọi là một **placeholder**), phép toán thứ hai biểu thị hằng số 3, phép toán thứ ba biểu thị phép toán lũy thừa (**), và phép toán cuối cùng biểu thị đầu ra của hàm này (nó là một phép toán đồng nhất (identity operation), nghĩa là nó sẽ không làm gì khác ngoài việc sao chép đầu ra của phép toán lũy thừa). Mỗi phép toán có một danh sách các tensor đầu vào và đầu ra mà bạn có thể dễ dàng truy cập bằng các thuộc tính `inputs` và `outputs` của phép toán. Ví dụ, hãy lấy danh sách các đầu vào và đầu ra của phép toán lũy thừa:

```
>>> pow_op = ops[2]

>>> list(pow_op.inputs)
[<tf.Tensor 'x:0' shape=() dtype=tf.float32>,
 <tf.Tensor 'pow/y:0' shape=() dtype=tf.float32>]

>>> pow_op.outputs
[<tf.Tensor 'pow:0' shape=() dtype=tf.float32>]
```

Đồ thị tính toán này được biểu diễn trong Hình D-2.



Hình D-2. Ví dụ về đồ thị tính toán.

Lưu ý rằng mỗi phép toán có một tên. Nó mặc định là tên của phép toán (ví dụ: “pow”), nhưng bạn có thể định nghĩa thủ công khi gọi phép toán (ví dụ: `tf.pow(x, 3, name="other_name")`). Nếu một tên đã tồn tại, TensorFlow tự động thêm một chỉ mục duy nhất (ví dụ: “pow_1”, “pow_2”, v.v.).

Mỗi tensor cũng có một tên duy nhất: nó luôn là tên của phép toán xuất ra tensor này, cộng với :0 nếu đó là đầu ra đầu tiên của phép toán, hoặc :1 nếu đó là đầu ra thứ hai, v.v. Bạn có thể lấy một phép toán hoặc một tensor bằng tên bằng cách sử dụng các phương thức `get_operation_by_name()` hoặc `get_tensor_by_name()` của đồ thị:

```
>>> concrete_function.graph.get_operation_by_name('x')
<tf.Operation 'x' type=Placeholder>

>>> concrete_function.graph.get_tensor_by_name('Identity:0')
<tf.Tensor 'Identity:0' shape=() dtype=tf.float32>
```

Hàm cụ thể cũng chứa định nghĩa hàm (được biểu diễn dưới dạng protocol buffer), bao gồm chữ ký của hàm. Chữ ký này cho phép hàm cụ thể biết các placeholder nào sẽ được cấp các giá trị đầu vào, và các tensor nào sẽ được trả về:

```
>>> concrete_function.function_def.signature
name: "inference_tf_cube_..." # Tên có thể khác nhau
input_arg {
  name: "x"
  type: DT_FLOAT
}
output_arg {
  name: "identity"
  type: DT_FLOAT
}
```

Bây giờ chúng ta hãy xem xét kỹ hơn về việc truy vết (tracing).

Xem xét kỹ hơn về Tracing

Hãy điều chỉnh hàm `tf_cube()` để in đầu vào của nó:

```
@tf.function
def tf_cube(x):
    print(f"x = {x}")
    return x ** 3
```

Bây giờ hãy gọi nó:

```
>>> result = tf_cube(tf.constant(2.0))
x = Tensor("x:0", shape=(), dtype=tf.float32)
llll
>>> result
<tf.Tensor: shape=(), dtype=tf.float32, numpy=8.0>
```

Kết quả trông tốt, nhưng hãy nhìn vào những gì đã được in: `x` là một tensor ký hiệu! Nó có hình dạng và kiểu dữ liệu, nhưng không có giá trị. Hơn nữa nó có một tên ("`x:0`"). Điều này là do hàm `print()` không phải là một phép toán TensorFlow, vì vậy nó sẽ chỉ chạy khi hàm Python được truy vết (traced), điều này xảy ra ở chế độ đồ thị, với các đối số được thay thế bằng các tensor ký hiệu (cùng kiểu và hình dạng, nhưng không có giá trị). Vì hàm `print()` không được ghi vào đồ thị, những lần tiếp theo chúng ta gọi `tf_cube()` với các tensor vô hướng float32, không có gì được in:

```
>>> result = tf_cube(tf.constant(3.0))
```

```
>>> result = tf_cube(tf.constant(4.0))
```

Nhưng nếu chúng ta gọi `tf_cube()` với một tensor có kiểu hoặc hình dạng khác, hoặc với một giá trị Python mới, hàm sẽ được truy vết lại, vì vậy hàm `print()` sẽ được gọi:

```
>>> result = tf_cube(2) # giá trị Python mới: truy vết!  
x = 2
```

```
>>> result = tf_cube(3) # giá trị Python mới: truy vết!  
x = 3
```

```
>>> result = tf_cube(tf.constant([[1., 2.]))) # hình dạng mới: truy vết!  
x = Tensor("x:0", shape=(1, 2), dtype=tf.float32)
```

```
>>> result = tf_cube(tf.constant([[3., 4.], [5., 6.]))) # hình dạng mới: truy  
vết!  
x = Tensor("x:0", shape=(None, 2), dtype=tf.float32)
```

```
>>> result = tf_cube(tf.constant([[7., 8.], [9., 10.]))) # cùng hình dạng: kh  
ông truy vết
```

Trong một số trường hợp, bạn có thể muốn hạn chế một hàm TF vào một chữ ký đầu vào cụ thể. Ví dụ, giả sử bạn biết rằng bạn sẽ chỉ gọi một hàm TF với các lô ảnh 28×28 pixel, nhưng các lô sẽ có kích thước rất khác nhau. Bạn có thể không muốn TensorFlow tạo ra một hàm cụ thể khác cho mỗi kích thước lô, hoặc dựa vào nó để tự mình tìm ra khi nào nên sử dụng None. Trong trường hợp này, bạn có thể chỉ định chữ ký đầu vào như sau:

```
@tf.function(input_signature=[tf.TensorSpec([None, 28, 28], tf.float32)])  
def shrink(images):  
    return images[:, ::2, ::2] # loại bỏ một nửa số hàng và cột
```

Hàm TF này sẽ chấp nhận bất kỳ tensor float32 nào có hình dạng `[*, 28, 28]`, và nó sẽ sử dụng lại cùng hàm cụ thể mọi lúc:

```
img_batch_1 = tf.random.uniform(shape=[100, 28, 28])  
img_batch_2 = tf.random.uniform(shape=[50, 28, 28])  
preprocessed_images = shrink(img_batch_1) # hoạt động tốt, truy vết hàm  
preprocessed_images = shrink(img_batch_2) # hoạt động tốt, cùng hàm cụ thể
```

Tuy nhiên, nếu bạn cố gắng gọi hàm TF này với một giá trị Python, hoặc một tensor có kiểu dữ liệu hoặc hình dạng không mong muốn, bạn sẽ nhận được một ngoại lệ:

```
img_batch_3 = tf.random.uniform(shape=[2, 2, 2])  
# preprocessed_images = shrink(img_batch_3) # ValueError! Đầu vào không tương  
thích (chạy dòng này sẽ gây lỗi)
```

Sử dụng AutoGraph để Ghi lại Luồng Điều khiển (Control Flow)

Nếu hàm của bạn chứa một vòng lặp for đơn giản, bạn mong đợi điều gì sẽ xảy ra? Ví dụ, hãy viết một hàm sẽ cộng 10 vào đầu vào của nó, bằng cách chỉ cộng 1 10 lần:

```
@tf.function
def add_10(x):
    for i in range(10):
        x += 1
    return x
```

Nó hoạt động tốt, nhưng khi chúng ta nhìn vào đồ thị của nó, chúng ta thấy rằng nó không chứa một vòng lặp: nó chỉ chứa 10 phép toán cộng!

```
>>> add_10(tf.constant(0))
<tf.Tensor: shape=(), dtype=tf.int32, numpy=10> # Kết quả là 10, không phải 1
5 như ví dụ gốc, do x ban đầu là 0

>>> add_10.get_concrete_function(tf.constant(0)).graph.get_operations()

# Output sẽ tương tự như sau (đã giản lược để dễ đọc):
# [<tf.Operation 'x' type=Placeholder>,
# <tf.Operation 'add' type=AddV2>,
# <tf.Operation 'add_1' type=AddV2>,
# ... (thêm 7 phép AddV2 nữa)
# <tf.Operation 'add_9' type=AddV2>,
# <tf.Operation 'Identity' type=Identity>]
```

Điều này thực sự có lý: khi hàm được truy vết, vòng lặp chạy 10 lần, vì vậy phép toán `x += 1` được chạy 10 lần, và vì nó ở chế độ đồ thị, nó đã ghi lại phép toán này 10 lần trong đồ thị. Bạn có thể nghĩ vòng lặp for này như một vòng lặp “tĩnh” được mở rộng (unrolled) khi đồ thị được tạo.

Nếu bạn muốn đồ thị chứa một vòng lặp “động” thay vào đó (tức là một vòng lặp chạy khi đồ thị được thực thi), bạn có thể tạo một cách thủ công bằng cách sử dụng phép toán `tf.while_loop()`, nhưng nó không trực quan lắm (xem phần “Using AutoGraph to Capture Control Flow” của sổ tay Chương 12 để biết ví dụ). Thay vào đó, đơn giản hơn nhiều là sử dụng tính năng **AutoGraph** của TensorFlow, đã thảo luận trong Chương 12.

AutoGraph thực sự được kích hoạt theo mặc định (nếu bạn cần tắt nó, bạn có thể truyền `autograph=False` cho `tf.function()`). Vậy nếu nó được bật, tại sao nó không ghi lại vòng lặp for trong hàm `add_10()`? Nó chỉ ghi lại các vòng lặp for lặp lại trên các tensor của đối tượng `tf.data.Dataset`, vì vậy bạn nên sử dụng `tf.range()`, không phải `range()`. Điều này là để cho bạn lựa chọn:

- Nếu bạn sử dụng `range()`, vòng lặp for sẽ là tĩnh, nghĩa là nó sẽ chỉ được thực thi khi hàm được truy vết. Vòng lặp sẽ được “mở rộng” thành một tập hợp các phép toán cho mỗi lần lặp, như chúng ta đã thấy.

- Nếu bạn sử dụng `tf.range()`, vòng lặp sẽ là động, nghĩa là nó sẽ được bao gồm trong chính đồ thị (nhưng nó sẽ không chạy trong quá trình truy vết).

Hãy xem đồ thị được tạo ra nếu chúng ta chỉ thay thế `range()` bằng `tf.range()` trong hàm `add_10()`:

```
@tf.function
def add_10_tf_range(x):
    for i in tf.range(10): # Thay đổi ở đây
        x += 1
    return x

>>> add_10_tf_range.get_concrete_function(tf.constant(0)).graph.get_operations()
# Output sẽ tương tự như sau (đã giản lược):
# [<tf.Operation 'x' type=Placeholder>,
#  <tf.Operation 'while' type=StatelessWhile>, ...]
```

Như bạn có thể thấy, đồ thị bây giờ chứa một phép toán vòng lặp `while`, như thể chúng ta đã gọi hàm `tf.whilst_loop()`.

Xử lý Biến và các Tài nguyên Khác trong Hàm TF

Trong TensorFlow, các biến và các đối tượng có trạng thái khác, chẳng hạn như hàng đợi hoặc tập dữ liệu, được gọi là **tài nguyên (resources)**. Các hàm TF xử lý chúng một cách đặc biệt: bất kỳ phép toán nào đọc hoặc cập nhật một tài nguyên đều được coi là **có trạng thái (stateful)**, và các hàm TF đảm bảo rằng các phép toán có trạng thái được thực thi theo thứ tự chúng xuất hiện (ngược lại với các phép toán không trạng thái, có thể chạy song song, vì vậy thứ tự thực thi của chúng không được đảm bảo). Hơn nữa, khi bạn truyền một tài nguyên làm đối số cho một hàm TF, nó được truyền theo tham chiếu (by reference), vì vậy hàm có thể sửa đổi nó. Ví dụ:

```
counter = tf.Variable(0)

@tf.function
def increment(counter_var, c=1): # Đổi tên counter thành counter_var để tránh
    # nhầm lẫn với biến global
    return counter_var.assign_add(c)

increment(counter) # counter là 1
increment(counter) # counter là 2
print(counter)
```

Nếu bạn nhìn vào định nghĩa hàm, đối số đầu tiên được đánh dấu là một tài nguyên:

```
>>> function_def = increment.get_concrete_function(counter).function_def
>>> function_def.signature.input_arg[0]
```

```
name: "counter_var"  
type: DT_RESOURCE
```

Cũng có thể sử dụng một `tf.Variable` được định nghĩa bên ngoài hàm, mà không cần truyền nó rõ ràng làm đối số:

```
global_counter = tf.Variable(0)  
  
@tf.function  
def increment_global(c=1):  
    return global_counter.assign_add(c)  
  
increment_global() # global_counter là 1  
increment_global() # global_counter là 2  
print(global_counter)
```

Hàm TF sẽ coi đây là một đối số đầu tiên ngầm định, vì vậy nó thực sự sẽ có cùng chữ ký (trừ tên đối số). Tuy nhiên, việc sử dụng biến toàn cục có thể nhanh chóng trở nên lộn xộn, vì vậy bạn thường nên gói các biến (và các tài nguyên khác) bên trong các lớp. Tin tốt là `@tf.function` cũng hoạt động tốt với các phương thức:

```
class MyCounter:  
    def __init__(self):  
        self.counter = tf.Variable(0)  
  
    @tf.function  
    def increment(self, c=1):  
        return self.counter.assign_add(c)  
  
my_obj_counter = MyCounter()  
my_obj_counter.increment()  
my_obj_counter.increment()  
print(my_obj_counter.counter)
```

Một ví dụ điển hình của cách tiếp cận hướng đối tượng này là Keras. Hãy xem cách sử dụng các hàm TF với Keras.

Sử dụng Hàm TF với Keras (hoặc không)

Theo mặc định, bất kỳ hàm, lớp hoặc mô hình tùy chỉnh nào bạn sử dụng với Keras sẽ tự động được chuyển đổi thành hàm TF; bạn không cần làm gì cả! Tuy nhiên, trong một số trường hợp, bạn có thể muốn hủy kích hoạt tính năng chuyển đổi tự động này — ví dụ, nếu mã tùy chỉnh của bạn không thể biến thành hàm TF, hoặc nếu bạn chỉ muốn gỡ lỗi mã của mình (để hơn nhiều ở chế độ `eager`). Để làm điều này, bạn có thể đơn giản truyền `dynamic=True` khi tạo mô hình hoặc bất kỳ lớp nào của nó:

```
# class MyModel(tf.keras.Model): ...  
# model = MyModel(dynamic=True)
```


Nếu mô hình hoặc lớp tùy chỉnh của bạn sẽ luôn là động, bạn có thể thay vào đó gọi hàm tạo của lớp cơ sở với `dynamic=True`:

```
class MyDense(tf.keras.layers.Layer):
    def __init__(self, units, **kwargs):
        super().__init__(dynamic=True, **kwargs)
        self.units = units
        # ... các khởi tạo khác ...

    def build(self, input_shape):
        self.w = self.add_weight(shape=(input_shape[-1], self.units),
                                initializer="random_normal",
                                trainable=True)
        self.b = self.add_weight(shape=(self.units,),
                                initializer="zeros",
                                trainable=True)
        super().build(input_shape)

    def call(self, inputs):
        return tf.matmul(inputs, self.w) + self.b
```

Ngoài ra, bạn có thể truyền `run_eagerly=True` khi gọi phương thức `compile()`:

```
# model.compile(loss=my_mse, optimizer="nadam", metrics=[my_mae], run_eagerly=True)
```

Bây giờ bạn đã biết cách các hàm TF xử lý tính đa hình (với nhiều hàm cụ thể), cách các đồ thị được tự động tạo bằng AutoGraph và truy vết, đồ thị trông như thế nào, cách khám phá các phép toán và tensor ký hiệu của chúng, cách xử lý các biến và tài nguyên, và cách sử dụng các hàm TF với Keras.

Phán đoán và Công thức Toán:

Phụ lục này tập trung vào cách TensorFlow xây dựng và quản lý các đồ thị tính toán thông qua cơ chế AutoGraph và Concrete Functions, cũng như cách xử lý các tài nguyên và luồng điều khiển. Mặc dù không có công thức toán học phức tạp được trình bày trực tiếp, các khái niệm liên quan đến đồ thị tính toán và biến đổi dữ liệu có thể được liên hệ với toán học.

- **Hàm `tf_cube(x)` = x^3 :** Đây là một phép toán lũy thừa cơ bản.
 - Đầu vào x có thể là một tensor (số vô hướng, vector, ma trận, tensor đa chiều).
 - Phép toán là nâng lũy thừa bậc ba cho từng phần tử của tensor đầu vào.
 - Ví dụ:
 - Nếu $x = 2.0$ (vô hướng), thì $x^3 = 2.0^3 = 8.0$.
 - Nếu $x = [2.0]$ (vector), thì $x^3 = [2.0^3] = [8.0]$.
 - Nếu $x = \begin{bmatrix} 1.0 & 2.0 \\ 3.0 & 4.0 \end{bmatrix}$ (ma trận), thì $x^3 = \begin{bmatrix} 1.0^3 & 2.0^3 \\ 3.0^3 & 4.0^3 \end{bmatrix} = \begin{bmatrix} 1.0 & 8.0 \\ 27.0 & 64.0 \end{bmatrix}$.

- **Phép toán `images[:, ::2, ::2]` trong hàm `shrink`:** Đây là một phép cắt lát (slicing) tensor trong NumPy/TensorFlow.
 - `::` chọn tất cả các phần tử dọc theo chiều đó.
 - `::2`: chọn các phần tử với bước nhảy là 2, tức là chỉ lấy các phần tử ở vị trí 0, 2, 4, ...
 - Với `images` có hình dạng `[batch_size, height, width]`, phép toán này sẽ giảm chiều cao và chiều rộng đi một nửa. Nếu `height` ban đầu là H và `width` ban đầu là W , thì chiều cao mới sẽ là $\lceil H/2 \rceil$ và chiều rộng mới là $\lceil W/2 \rceil$.

$$shape_{new} = (batch_size, \lceil height/2 \rceil, \lceil width/2 \rceil)$$
 - Ví dụ: Nếu `images` có shape `[100, 28, 28]`, `shrink(images)` sẽ trả về một tensor có shape `[100, 14, 14]`.
- **Vòng lặp `x += 1` trong đồ thị tĩnh:** Khi `for i in range(10): x += 1` được chuyển đổi thành đồ thị tĩnh, nó thực chất là một chuỗi 10 phép toán cộng:

$$x_{\{new\}} = \vdash(\vdash(\vdash(\vdash(\vdash(\vdash(\vdash(x_{\{initial\}} + 1) + 1) + 1) + 1) + 1) + 1) + 1) + 1) + 1) + 1)$$

 Đây là một phép toán đơn giản: $x_{new} = x_{initial} + 10$.

Các khái niệm về FuncGraph, Operation, Tensor đều là các thành phần của một biểu đồ tính toán. Biểu đồ này về cơ bản biểu diễn một chuỗi các phép toán toán học (ví dụ: lũy thừa, cộng, nhân ma trận) và luồng dữ liệu (tensor) giữa chúng. Mục tiêu của việc chuyển đổi sang đồ thị là để tối ưu hóa việc thực thi các phép toán này, ví dụ như song song hóa, tối ưu hóa bộ nhớ, hoặc biên dịch thành mã máy hiệu quả hơn.